DBMS ASSIGNMENT SERIES

Assignment 6

Implementation of Data Integrity Constraints

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Define rules	Apply constraints	Verify data	Test failures

1. Objective

Constraints prevent invalid records from entering the database. The primary and foreign keys are created in Assignment 5; this stage adds uniqueness, value-range checks, and defaults. The script targets MySQL 8.0.16 or later so CHECK constraints are enforced.

2. Constraint Inventory

Constraint type	Where used	Purpose
PRIMARY KEY	All eight tables; composite (Order_ID, Item_ID) in Order_Item	Uniquely identifies a row and prevents duplicate primary-key values.
FOREIGN KEY	Address, Menu_Item, Orders, Order_Item, Payment	Requires referenced parent rows to exist and protects relationships.
UNIQUE	Customer.Email, Customer.Phone, Restaurant.Phone, Delivery_Partner.Phone, Vehicle_Number, Payment.Order_ID	Prevents duplicate contact/vehicle identifiers and enforces at most one payment row per order.
CHECK	Rating, menu price/availability, delivery status, order status, quantity, unit price, payment method/status/date	Restricts values to valid business ranges and supported states.
DEFAULT	Restaurant.Rating, Menu_Item.Availability, Delivery_Partner.Status, Orders.Order_Date/Order_Status, Order_Item.Quantity, Payment.Payment_Status	Supplies sensible values when an insert omits an optional value.

3. Important Constraint Examples

```
ALTER TABLE Customer
  ADD CONSTRAINT uq_customer_email UNIQUE (Email),
  ADD CONSTRAINT uq_customer_phone UNIQUE (Phone);

ALTER TABLE Restaurant
  ALTER COLUMN Rating SET DEFAULT 0.0,
  ADD CONSTRAINT ck_restaurant_rating CHECK (Rating BETWEEN 0.0 AND 5.0);

ALTER TABLE Order_Item
  ALTER COLUMN Quantity SET DEFAULT 1,
  ADD CONSTRAINT ck_order_item_quantity CHECK (Quantity > 0);

ALTER TABLE Payment
  ADD CONSTRAINT uq_payment_order UNIQUE (Order_ID),
  ADD CONSTRAINT ck_payment_status CHECK
    (Payment_Status IN ('Pending', 'Paid', 'Failed', 'Refunded'));
```

4. Verification Queries

Run the SELECT checks in Assignment_6_Constraint_Verification.sql after Assignment 7 inserts the sample rows. Every count below is expected to be zero, meaning no invalid records were found. The displayed counts were checked against the bundled fictional dataset; run the supplied MySQL 8.x script in Workbench for native execution evidence.

Verification result	Expected value	Meaning
Duplicate email groups	0	No email occurs more than once.
Duplicate customer phone groups	0	No customer phone occurs more than once.
Invalid ratings	0	Every restaurant rating is between 0 and 5.
Invalid menu prices	0	No menu item has a negative price.
Invalid quantities	0	Every stored order quantity is greater than zero.
Orders using another customer's address	0	The composite foreign key and sample data preserve address ownership.

5. Negative Test Cases

The following statements are intentionally commented out in the SQL file. Run each one separately only after loading sample data; each should be rejected. Do not uncomment them in the full setup script.

```
-- Duplicate email/phone: expected UNIQUE constraint error
-- INSERT INTO Customer (Name, Email, Phone, Password_Hash)
-- VALUES ('Duplicate Test', 'asha@example.com', '9000000001', 'demo_hash');

-- Rating outside the allowed range: expected CHECK error
-- INSERT INTO Restaurant (Restaurant_Name, Phone, Address, Rating)
-- VALUES ('Invalid Rating Test', '9000000999', 'Test Street', 6.0);

-- Zero quantity: expected CHECK error
-- INSERT INTO Order_Item (Order_ID, Item_ID, Quantity, Unit_Price)
-- VALUES (1, 1, 0, 220.00);

-- Paid without a payment date: expected CHECK error
-- INSERT INTO Payment (Order_ID, Payment_Method, Payment_Status, Payment_Date)
-- VALUES (1, 'UPI', 'Paid', NULL);
```

6. Why These Constraints Matter

Primary and foreign keys preserve identity and links; UNIQUE prevents ambiguous accounts and payments; CHECK blocks values that the business process cannot accept; DEFAULT reduces repeated boilerplate while providing standard initial states. Constraints do not replace application validation, transaction handling, or appropriate access control, but they provide an essential database-level safety net.